

Scripts Execution

Explanation of the solution to the streaming layer problem

- **Task 5: Create a streaming data processing framework that ingests real-time POS transaction data from Kafka. The transaction data is then validated based on the three rules' parameters (stored in the NoSQL database) discussed previously.**
- **Task 6: Update the transactions data along with the status (fraud/genuine) in the card_transactions table.**
- **Task 7: Store the 'postcode' and 'transaction_dt' of the current transaction in the look-up table in the NoSQL database if the transaction was classified as genuine.**

```

root@ip-10-0-3-91:~#
Verifying : python-rpm-macros-3-60.amzn2.0.1.noarch
Verifying : python3-devel-3.7.16-1.amzn2.0.1.x86_64
Verifying : python3-rpm-macros-3-60.amzn2.0.1.noarch
Verifying : python-srpm-macros-3-60.amzn2.0.1.noarch

Installed:
python3-devel.x86_64 0:3.7.16-1.amzn2.0.1

Dependency Installed:
python-rpm-macros.noarch 0:3-60.amzn2.0.1          python-srpm-macros.noarch 0:3-60.amzn2.0.1          python3-rpm-macros.noarch 0:3-60.amzn2.0.1

Complete!
[root@ip-10-0-3-91 ~]# pip install happybase
WARNING: Running pip install with root privileges is generally not a good idea. Try 'pip3 install --user' instead.
Collecting happybase
  Downloading happybase-1.2.0.tar.gz (40 kB)
    | 40 kB 7.1 MB/s
Requirement already satisfied: six in /usr/local/lib/python3.7/site-packages (from happybase) (1.13.0)
Collecting thriftpy2>=0.4
  Downloading thriftpy2-0.4.16.tar.gz (643 kB)
    | 643 kB 50.3 MB/s
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
    | 49 kB 10.5 MB/s
Using legacy 'setup.py install' for happybase, since package 'wheel' is not installed.
Using legacy 'setup.py install' for thriftpy2, since package 'wheel' is not installed.
Installing collected packages: ply, thriftpy2, happybase
  Running setup.py install for thriftpy2 ... done
  Running setup.py install for happybase ... done
ERROR: After October 2020 you may experience errors when installing or updating packages. This is because pip will change the way that it resolves dependency conflicts.

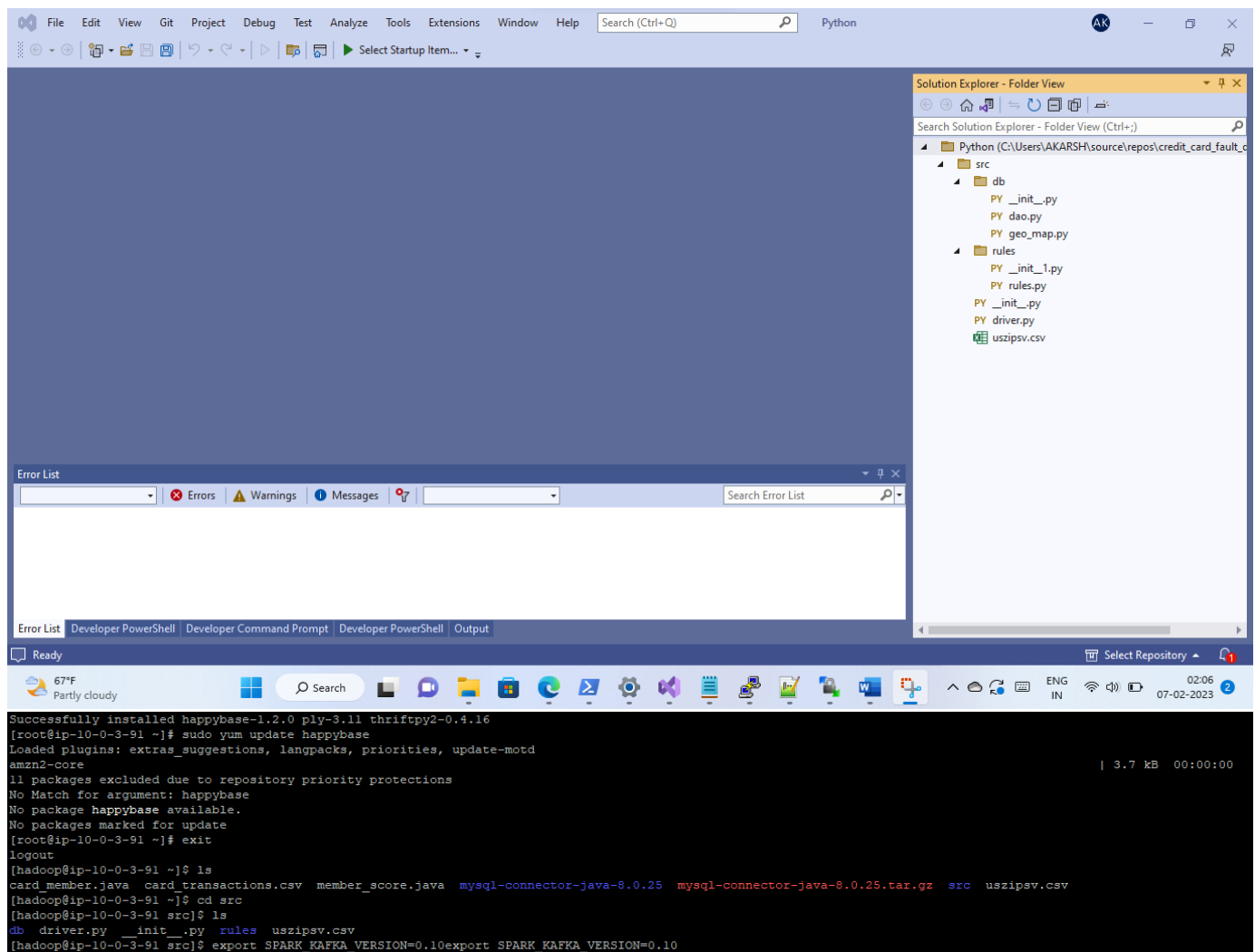
We recommend you use --use-feature=2020-resolver to test your packages with the new resolver before it becomes the default.

thriftpy2 0.4.16 requires six<=1.15, but you'll have six 1.13.0 which is incompatible.
Successfully installed happybase-1.2.0 ply-3.11 thriftpy2-0.4.16
[root@ip-10-0-3-91 ~]# sudo yum update happybase
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core                               | 3.7 kB  00:00:00
11 packages excluded due to repository priority protections
No Match for argument: happybase
No package happybase available.
No packages marked for update
[root@ip-10-0-3-91 ~]#

```

1. Logged into EMR instance as “hadoop”:
2. Run the following commands in order to Install Happy base and start thrift server
 - `sudo yum update`
 - `sudo yum install python3-devel`
 - `pip install happybase`
 - `/usr/lib/hbase/bin/hbase-daemon.sh start thrift -p 9090`

- Downloaded db-> dao.py , geomap.py ,rules-> rules.py ,driver.py ,unzipsv.csv from the resource section of the capstone project from the learning platform and transfer it to hadoopinstance via WinSCP.



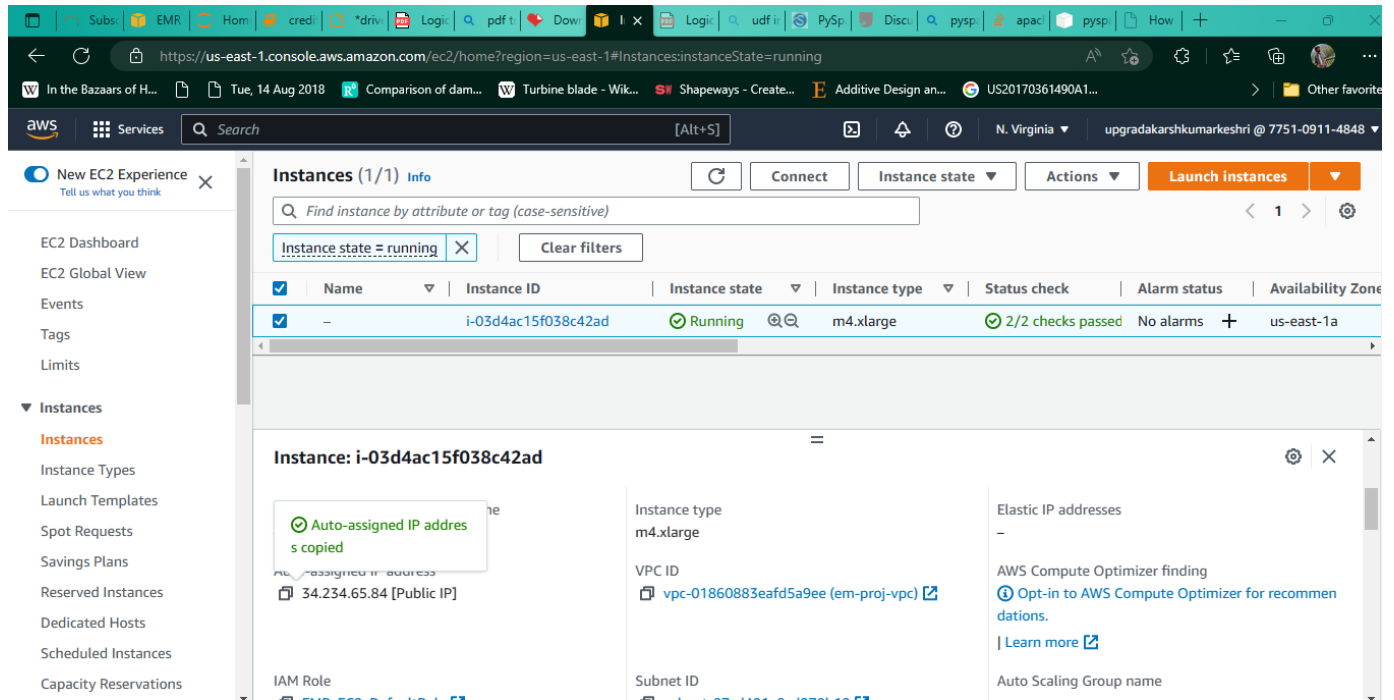
The screenshot shows a Visual Studio Code interface with a Python project. The Solution Explorer on the right displays the following file structure:

- Python (C:\Users\AKARSH\source\repos\credit_card_fault_d
 - src
 - db
 - PY __init__.py
 - PY dao.py
 - PY geo_map.py
 - rules
 - PY __init__.py
 - PY rules.py
 - PY __init__.py
 - PY driver.py
 - uszipsv.csv

The terminal at the bottom shows the following commands and output:

```
Successfully installed happybase-1.2.0 ply-3.11 thriftpy2-0.4.16
[root@ip-10-0-3-91 ~]# sudo yum update happybase
Loaded plugins: extras_suggestions, langpacks, priorities, update-mo
amzn2-core
11 packages excluded due to repository priority protections
No Match for argument: happybase
No package happybase available.
No packages marked for update
[root@ip-10-0-3-91 ~]# exit
logout
[hadoop@ip-10-0-3-91 ~]$ ls
card_member.java  card_transactions.csv  member_score.java  mysql-connector-java-8.0.25  mysql-connector-java-8.0.25.tar.gz  src  uszipsv.csv
[hadoop@ip-10-0-3-91 ~]$ cd src
[hadoop@ip-10-0-3-91 src]$ ls
db  driver.py  __init__.py  rules  uszipsv.csv
[hadoop@ip-10-0-3-91 src]$ export SPARK_KAFKA_VERSION=0.10export SPARK_KAFKA_VERSION=0.10
```

4. Updated the Public IP of your EC2 Instance “34.234.65.84”(self.host) in dao.py file



The screenshot displays the AWS Management Console interface for the EC2 service. The left sidebar shows the navigation menu with options like EC2 Dashboard, EC2 Global View, Events, Tags, Limits, and Instances. The main content area shows the 'Instances (1/1) Info' page. A table lists the instance details:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
-	i-03d4ac15f038c42ad	Running	m4.xlarge	2/2 checks passed	No alarms	us-east-1a

Below the table, the 'Instance: i-03d4ac15f038c42ad' details are shown. A green notification box states 'Auto-assigned IP address copied' with the IP address 34.234.65.84 [Public IP]. Other details include:

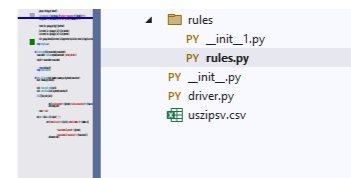
- Instance type: m4.xlarge
- VPC ID: vpc-01860883eafd5a9ee (em-proj-vpc)
- Subnet ID: subnet-01860883eafd5a9ee
- Elastic IP addresses: -
- AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendations. (Learn more)
- Auto Scaling Group name: -

- Updated **rules.py** with following parameters:
 lookup_table =
 'lookup_data_hbase'
 master_table =
 'Hbase_card_transaction'

```
"""
from db_dao import HBaseDao
from db_dao import GEO_Map
from datetime import datetime
import uuid

lookup_table = 'lookup_data_hbase'
master_table = 'card_transactions_hbase'

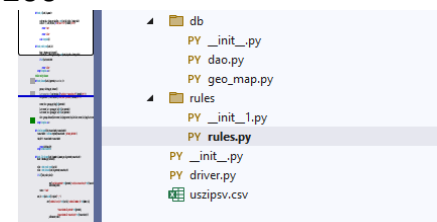
# Rule - 1
```



- Created Python functions, containing the logic for the UDFs (**rules.py**)
 check_ucl : Function to verify the UCL rule Transaction amount should be less than Uppercontrol limit (UCL)

- verify_credit_score_data: Function to verify the credit score rule
 .Credit score of each membershould be greater than 200

```
def check_ucl(card_id,amount):
    try:
        hbase = HBaseDao.get_instance()
        card_row_data = hbase.get_data(key = str(card_id),table = lookup_table)
        card_ucl = ( card_row_data [b'card_data':'ucl']).decode("utf-8")
        if amount < float(card_ucl):
            return 'True'
        else:
            return 'False'
    except Exception as e:
        raise Exception(e)
```



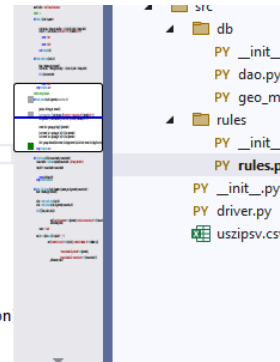
- Check_distance: Function to verify the following zipcode rules. ZIP code distance

```
def check_distance(card_id, postcode, transaction_dt):
    try:
        hbase = HBaseDao.get_instance()
        geo_map = GEO_Map.get_instance()
        card_row_data = hbase.get_data(key = str(card_id), table = lookup_table)
        last_transaction = ( card_row_data [b'card_data':'transaction_dt']).decode("utf-8")
        last_postcode = ( card_row_data [b'card_data':'postcode']).decode("utf-8")

        current_lat = geo_map.get_lat(str(postcode))
        current_lon = geo_map.get_long(str(postcode))

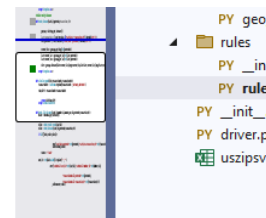
        last_current_lat = geo_map.get_lat(str(last_postcode))
        last_current_lon = geo_map.get_long(str(last_postcode))

        dist = geo_map.distance(lat1=current_lat, long1=current_lon, lat2=last_current_lat, long2=last_current_lon)
        speed = calculate_speed(dist, transactiondt1, transactiondt2)
    except Exception as e:
```

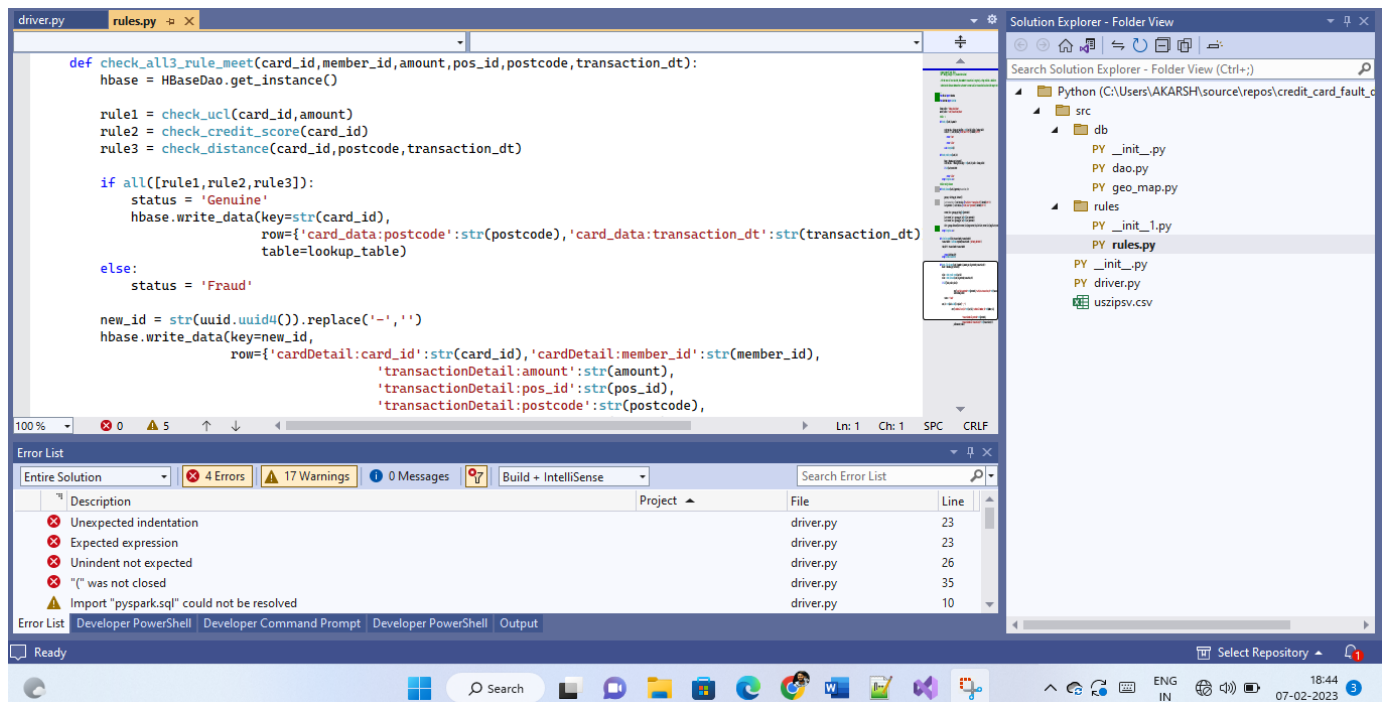


- calculate_speed : A function to calculate the speed from distance and transaction timestamp differentials

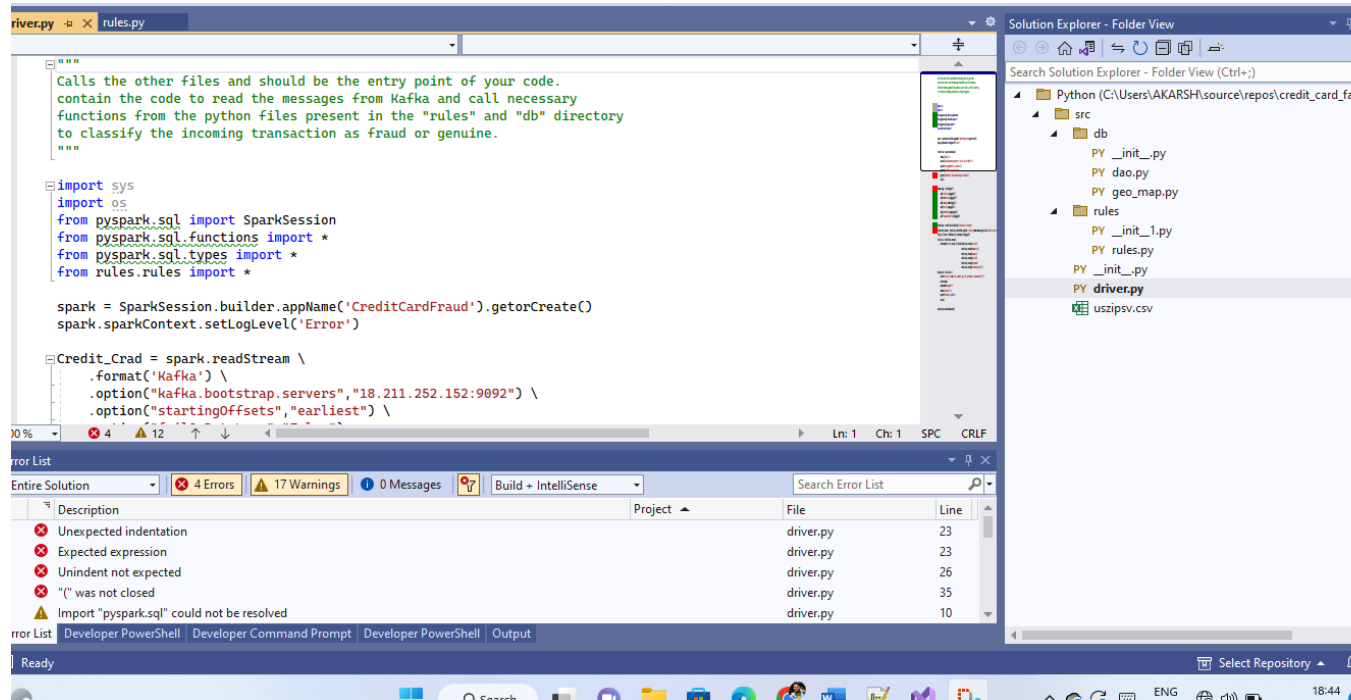
```
def calculate_speed(dist, transactiondt1, transactiondt2):
    transactiondt1 = datetime.strptime(transactiondt1, '%d-%m-%y %H:%M:%S')
    transactiondt2 = datetime.strptime(transactiondt2, '%d-%m-%y %H:%M:%S')
    time_diff = transactiondt1 - transactiondt2
    time_diff = time_diff.total_seconds()
    try:
        return dist/time_diff
    except ZeroDivisionError:
        return '300000'
```



Check_all3_rule_meet: A function to verify all the three rules - ucl, credit score and speed



- Next, I updated the 'driver.py' file with the following code
Setting up the system dependencies and importing necessary libraries and modules



```

"""
Calls the other files and should be the entry point of your code.
contain the code to read the messages from Kafka and call necessary
functions from the python files present in the "rules" and "db" directory
to classify the incoming transaction as fraud or genuine.
"""

import sys
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from rules.rules import *

spark = SparkSession.builder.appName('CreditCardFraud').getOrCreate()
spark.sparkContext.setLogLevel('Error')

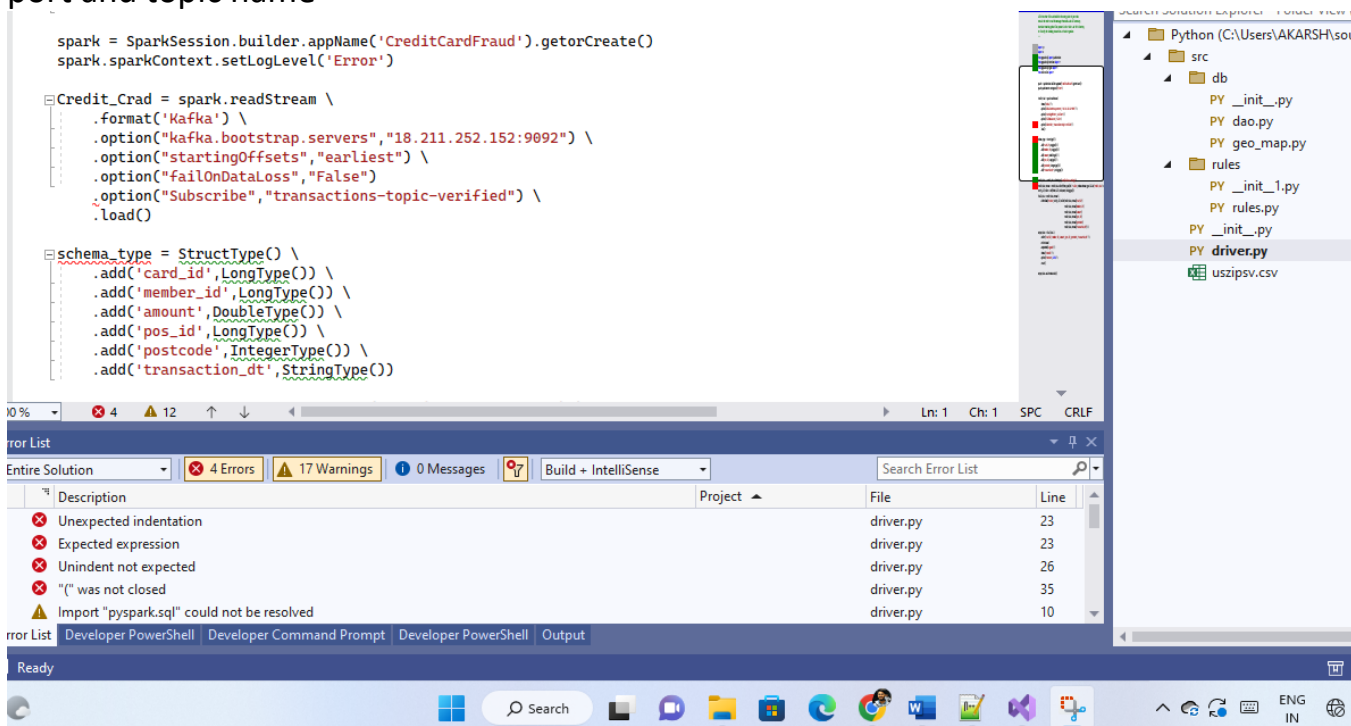
Credit_Crad = spark.readStream \
    .format('Kafka') \
    .option("kafka.bootstrap.servers", "18.211.252.152:9092") \
    .option("startingOffsets", "earliest") \
    .load()

schema_type = StructType() \
    .add('card_id', LongType()) \
    .add('member_id', LongType()) \
    .add('amount', DoubleType()) \
    .add('pos_id', LongType()) \
    .add('postcode', IntegerType()) \
    .add('transaction_dt', StringType())

```

Description	Project	File	Line
Unexpected indentation		driver.py	23
Expected expression		driver.py	23
Unindent not expected		driver.py	26
"(" was not closed		driver.py	35
Import "pyspark.sql" could not be resolved		driver.py	10

- Initializing the Spark session and reading input data from Kafka mentioning the details of the Kafka broker, such as bootstrap server, port and topic name



```

spark = SparkSession.builder.appName('CreditCardFraud').getOrCreate()
spark.sparkContext.setLogLevel('Error')

Credit_Crad = spark.readStream \
    .format('Kafka') \
    .option("kafka.bootstrap.servers", "18.211.252.152:9092") \
    .option("startingOffsets", "earliest") \
    .option("failOnDataLoss", "False") \
    .option("Subscribe", "transactions-topic-verified") \
    .load()

schema_type = StructType() \
    .add('card_id', LongType()) \
    .add('member_id', LongType()) \
    .add('amount', DoubleType()) \
    .add('pos_id', LongType()) \
    .add('postcode', IntegerType()) \
    .add('transaction_dt', StringType())

```

Description	Project	File	Line
Unexpected indentation		driver.py	23
Expected expression		driver.py	23
Unindent not expected		driver.py	26
"(" was not closed		driver.py	35
Import "pyspark.sql" could not be resolved		driver.py	10

- Connect to kafka topic

using Bootstrap-server:
18.211.252.152 Port
Number: 9092
Topic: transactions-topic-verified

11. Define JSON schema of each transactions

```
schema_type = StructType() \
    .add('card_id', LongType()) \
    .add('member_id', LongType()) \
    .add('amount', DoubleType()) \
    .add('pos_id', LongType()) \
    .add('postcode', IntegerType()) \
    .add('transaction_dt', StringType())

credit_data = credit_data.selectExpr("cast(value as string)")
credit_data_stream = credit_data.select(from_json(col='value', schema=schema_type).alias("credit_data").select(
    verify_all_rules = udf(check_all3_rule_meet, StringType())
final_data = credit_data_stream \
    .withcolumn('status', verify_all_rules(credit_data_stream['card_id'],
                                         credit_data_stream['member_id'],
                                         credit_data_stream['amount']
```

12. Read the raw JSON data from Kafka as 'credit_data_stream' and Define UDF's to verify rules

```
# Casting raw data as string and aliasing
credit_data = credit_data.selectExpr("cast(value as string)")
credit_data_stream = credit_data.select(from_json(col="value", schema=dataSchema).alias("credit_data")).select(
    "credit_data.*")

# Define UDF which verifies all the rules for each transaction and updates the lookup and master tables
verify_all_rules = udf(verify_rules_status, StringType())

Final_data = credit_data_stream \
    .withColumn('status', verify_all_rules(credit_data_stream['card_id'],
                                         credit_data_stream['member_id'],
                                         credit_data_stream['amount'],
                                         credit_data_stream['pos_id'],
                                         credit_data_stream['postcode'],
                                         credit_data_stream['transaction_dt']))
```

13. Code to display output in console

```
credit_data_stream['pos_id']
credit_data_stream['postcode']
credit_data_stream['transaction_dt'])])

output_data = final_data \
    .select("card_id", "member_id", "amount", "pos_id", "postcode", "transaction_dt") \
    .writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", False) \
    .start()

output_data.awaitTermination()
```

14. Define spark termination

15. Set the Kafka Version using the following command
export SPARK_KAFKA_VERSION=0.10

16. Run the spark-submit command, specifying the Spark-SQL-Kafka package and python file

 hadoop@ip-172-31-23-33:~/python/src

```
[hadoop@ip-172-31-23-33 src]$ export SPARK_KAFKA_VERSION=0.10
```

```
[hadoop@ip-172-31-23-33 src]$ spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.11:2.4.5 driver.py
```

spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.11:2.4.5
driver.py

```
hadoop@ip-10-0-3-91:~/src
tracking URL: http://ip-10-0-3-91.ec2.internal:20888/proxy/application_1675660356305_0014/
user: hadoop
23/02/06 20:04:22 INFO Client: Application report for application_1675660356305_0014 (state: ACCEPTED)
23/02/06 20:04:23 INFO Client: Application report for application_1675660356305_0014 (state: ACCEPTED)
23/02/06 20:04:24 INFO Client: Application report for application_1675660356305_0014 (state: ACCEPTED)
23/02/06 20:04:25 INFO Client: Application report for application_1675660356305_0014 (state: ACCEPTED)
23/02/06 20:04:26 INFO Client: Application report for application_1675660356305_0014 (state: RUNNING)
23/02/06 20:04:26 INFO Client:
  client token: N/A
  diagnostics: N/A
  ApplicationMaster host: 10.0.3.91
  ApplicationMaster RPC port: -1
  queue: default
  start time: 1675713860794
  final status: UNDEFINED
  tracking URL: http://ip-10-0-3-91.ec2.internal:20888/proxy/application_1675660356305_0014/
  user: hadoop
23/02/06 20:04:26 INFO YarnClientSchedulerBackend: Application application_1675660356305_0014 has started running.
23/02/06 20:04:26 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 35917.
23/02/06 20:04:26 INFO NettyBlockTransferService: Server created on ip-10-0-3-91.ec2.internal:35917
23/02/06 20:04:26 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
23/02/06 20:04:27 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, ip-10-0-3-91.ec2.internal, 35917, None)
23/02/06 20:04:27 INFO BlockManagerMasterEndpoint: Registering block manager ip-10-0-3-91.ec2.internal:35917 with 1028.8 MB RAM, BlockManagerId(driver, ip-10-0-3-91.ec2.internal, 35917, None)
23/02/06 20:04:27 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, ip-10-0-3-91.ec2.internal, 35917, None)
23/02/06 20:04:27 INFO BlockManager: external shuffle service port = 7337
23/02/06 20:04:27 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, ip-10-0-3-91.ec2.internal, 35917, None)
23/02/06 20:04:27 INFO YarnClientSchedulerBackend: Add WebUI Filter. org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter, Map(PROXY_HOSTS -> ip-10-0-3-91.ec2.internal, PROXY_URI BASES -> http://ip-10-0-3-91.ec2.internal:20888/proxy/application_1675660356305_0014, /proxy/application_1675660356305_0014)
23/02/06 20:04:27 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter to /metrics/json.
23/02/06 20:04:27 INFO YarnSchedulerBackend$YarnSchedulerEndpoint: ApplicationMaster registered as NettyRpcEndpointRef(spark-client://YarnRM)
23/02/06 20:04:27 INFO EventLoggingListener: Logging events to hdfs://var/log/spark/apps/application_1675660356305_0014
23/02/06 20:04:27 INFO Utils: Using initial executors = 50, max of spark.dynamicAllocation.initialExecutors, spark.dynamicAllocation.minExecutors and spark.executor.instances
23/02/06 20:04:27 INFO YarnClientSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.0
23/02/06 20:04:28 INFO SharedState: loading hive config file: file:/etc/spark/conf/dist/hive-site.xml
23/02/06 20:04:28 INFO SharedState: Setting hive.metastore.warehouse.dir ('null') to the value of spark.sql.warehouse.dir ('hdfs://user/spark/warehouse').
23/02/06 20:04:28 INFO SharedState: Warehouse path is 'hdfs://user/spark/warehouse'.
23/02/06 20:04:28 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter to /SQL.
23/02/06 20:04:28 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter to /SQL/json.
23/02/06 20:04:28 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter to /SQL/execution.
23/02/06 20:04:28 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter to /SQL/execution/json.
23/02/06 20:04:28 INFO JettyUtils: Adding filter org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter to /static/sql.
```

17. Check Output in console :

```

=====
-----+-----+-----+-----+-----+-----+-----+-----+
card_id      |member_id|amount|pos_id|postcode|transaction dt ts|status|
-----+-----+-----+-----+-----+-----+-----+-----+
348702330256514|37495066290|4380912|248063406800722|96774|2017-12-31 08:24:29|GENUINE|
348702330256514|37495066290|6703385|786562777140812|84758|2017-12-31 04:15:03|FRAUD |
348702330256514|37495066290|7454328|466952571393508|93645|2017-12-31 09:56:42|GENUINE|
348702330256514|37495066290|4013428|45845320330319|15868|2017-12-31 05:38:54|GENUINE|
348702330256514|37495066290|5495353|545499621965697|79033|2017-12-31 21:51:54|GENUINE|
348702330256514|37495066290|3966214|369266342272501|22832|2017-12-31 03:52:51|GENUINE|
348702330256514|37495066290|1753644|9475029292671|17923|2017-12-31 00:11:30|FRAUD |
348702330256514|37495066290|1692115|27647525195860|55708|2017-12-31 17:02:39|GENUINE|
5189563368503974|117826301530|9222134|525701337355194|64002|2017-12-31 20:22:10|GENUINE|
5189563368503974|117826301530|4133848|182031383443115|26346|2017-12-31 01:52:32|FRAUD |
5189563368503974|117826301530|8938921|799748246411019|76934|2017-12-31 05:20:53|FRAUD |
5189563368503974|117826301530|1786366|131276818071265|63431|2017-12-31 14:29:38|GENUINE|
5189563368503974|117826301530|9142237|564240259678903|50635|2017-12-31 19:37:19|GENUINE|
5407073344486464|1147922084344|6885448|887913906711117|59031|2017-12-31 07:53:53|FRAUD |
5407073344486464|1147922084344|4028209|116266051118182|80118|2017-12-31 01:06:50|FRAUD |
5407073344486464|1147922084344|3858369|896105817613325|53820|2017-12-31 17:37:26|GENUINE|
5407073344486464|1147922084344|9307733|729374116016479|14898|2017-12-31 04:50:16|FRAUD |
5407073344486464|1147922084344|4011296|543373367319647|44028|2017-12-31 13:09:34|GENUINE|
5407073344486464|1147922084344|9492531|211980095659371|49453|2017-12-31 14:12:26|GENUINE|
5407073344486464|1147922084344|7550074|345533088112099|15030|2017-12-31 02:34:52|FRAUD |
=====
only showing top 20 rows

```

18.Count Data in Hbase: `count 'lookup_data_hive'`

```

current count: 20000, row: 27999
current count: 21000, row: 28999
current count: 22000, row: 29999
current count: 23000, row: 30999
current count: 24000, row: 31999
current count: 25000, row: 32999
current count: 26000, row: 33999
current count: 27000, row: 341724964458387.210778177559185.12-06-2018152438.2021-01-04171328.398477
current count: 28000, row: 346618652451637.549752175694218.28-09-2018005259.2021-01-04171400.227029
current count: 29000, row: 35264
current count: 30000, row: 36164
current count: 31000, row: 370552035866789.439446468625434.08-07-2018094337.2021-01-04171349.489439
current count: 32000, row: 375804375521605.880937166405449.26-05-2018130045.2021-01-04171430.733012
current count: 33000, row: 38176
current count: 34000, row: 39076
current count: 35000, row: 39977
current count: 36000, row: 40748
current count: 37000, row: 41560
current count: 38000, row: 42397
current count: 39000, row: 4315541850434035.496412742732167.12-02-2018148807.2021-01-04171356.009418
current count: 40000, row: 43999
current count: 41000, row: 44784
current count: 42000, row: 45546
current count: 43000, row: 46304
current count: 44000, row: 47134
current count: 45000, row: 47925
current count: 46000, row: 48730
current count: 47000, row: 49500
current count: 48000, row: 50351
current count: 49000, row: 5120
current count: 50000, row: 51988
current count: 51000, row: 5257502990314019.205172644364018.14-07-2018070014.2021-01-04171327.647742
current count: 52000, row: 53290
current count: 53000, row: 5420
current count: 54000, row: 6211
current count: 55000, row: 6478888641720566.273246841077378.06-10-2018212851.2021-01-04171335.535877
current count: 56000, row: 6968
current count: 57000, row: 7868
current count: 58000, row: 8768
current count: 59000, row: 9668
59367 row(s) in 3.8140 seconds
> 59367

```

Total number for record is **59367** which is matching with given requirement